



Svelte Review Cards

Svelte 5 review cards. Read the question, cover the answer, answer aloud, then check the code and the figure.

The Compiler Architecture

Q01

SVELTE 5 COMPILER VIRTUAL DOM

INTERVIEW QUESTION

Question. What is Svelte, and how does its architectural approach differ from Virtual DOM-based frameworks like React or Vue?

ANSWER.

The Runtime Tax

- **Heavy Engines:** traditional frameworks ship a heavy runtime library to the browser to manage the UI.
- **Costly Calculations:** when state changes, they generate a new Virtual DOM tree, diff it against the old one, and then apply changes to the real DOM, which costs valuable CPU cycles on every interaction.

Shifting Work to Compile Time

- **No Live Engine:** Svelte is a compiler. It does not ship a heavy framework engine to the browser.
- **Static Analysis:** it parses your `.svelte` components at compile time, analyzing exactly how your state maps to your markup before the code ever reaches the user.

Surgical DOM Updates

- **Imperative Output:** instead of relying on a Virtual DOM diffing engine, Svelte generates lean, imperative JavaScript.
- **Instant Execution:** when state changes, the compiled code already knows where to go. It updates only the specific text node or attribute, surgically and instantly.

Counter.svelte

```
<script>
  // ✓ The compiler tracks this **STATE** automatically
  let count = $state(0);

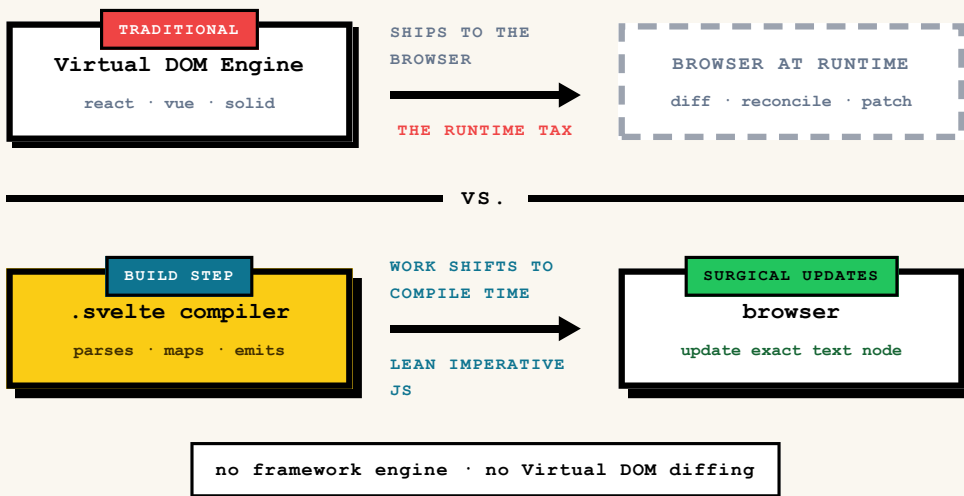
  function increment() {
    count += 1;
  }
</script>

<!-- ✓ Svelte wires the event directly to the **DOM** -->
<button onclick={increment}>
```

```
Clicked {count} times  
</button>
```

❶ **Summary.** Svelte is a compiler that shifts the heavy lifting from the browser's runtime to the build step. It translates declarative components into imperative JavaScript that surgically updates the DOM, entirely eliminating the overhead of the Virtual DOM.

COMPILER ARCHITECTURE



Inside the Build Step

Q02

SVELTE 5 COMPILER BUILD STEP AST

INTERVIEW QUESTION

Question. Explain what it means that Svelte is a compiler. What happens during the build step?

ANSWER.

Phase 1: Parsing and Analysis

- **AST Creation:** the compiler breaks your `.svelte` file down into an Abstract Syntax Tree (AST).
- **Reactivity Mapping:** it maps the relationships between reactive data (like the `$state` rune) and the HTML markup so it knows exactly what to update.

Phase 2: CSS Extraction and Scoping

- **Scoped Hashes:** styles are scoped to the component by appending a unique hash (e.g., `svelte-xyz123`) to the HTML elements and CSS selectors.
- **Extraction:** the CSS is extracted and bundled into a highly optimized, standalone `.css` file for the browser.

Phase 3: JavaScript Generation

- **Stripping Syntax:** the compiler strips away Svelte-specific syntax and HTML.
- **Imperative Lifecycle:** it generates pure, vanilla, imperative JavaScript instructions to handle the component's complete lifecycle: mounting physical DOM nodes, updating specific text nodes, and destroying

them cleanly.

Conceptual_Compiled_Output.js

```
const h1 = document.createElement('h1'); // ✓ 1. the **DOM**  
element is created via JavaScript  
h1.className = 'svelte-xyz123'; // ✓ 2. CSS is **SCOPED** with  
a hash and extracted to bundle.css  
function update_name(new_name) { // ✓ 3. the exact update  
instruction is **HARDCODED**  
  h1.textContent = `Hello ${new_name}!`;  
}
```

① **Summary.** Saying Svelte is a compiler means it translates `.svelte` files into standard web assets at build time. It parses the AST to map reactivity, scopes and extracts CSS, and generates lean, imperative JavaScript instructions for the browser to execute without a framework runtime.

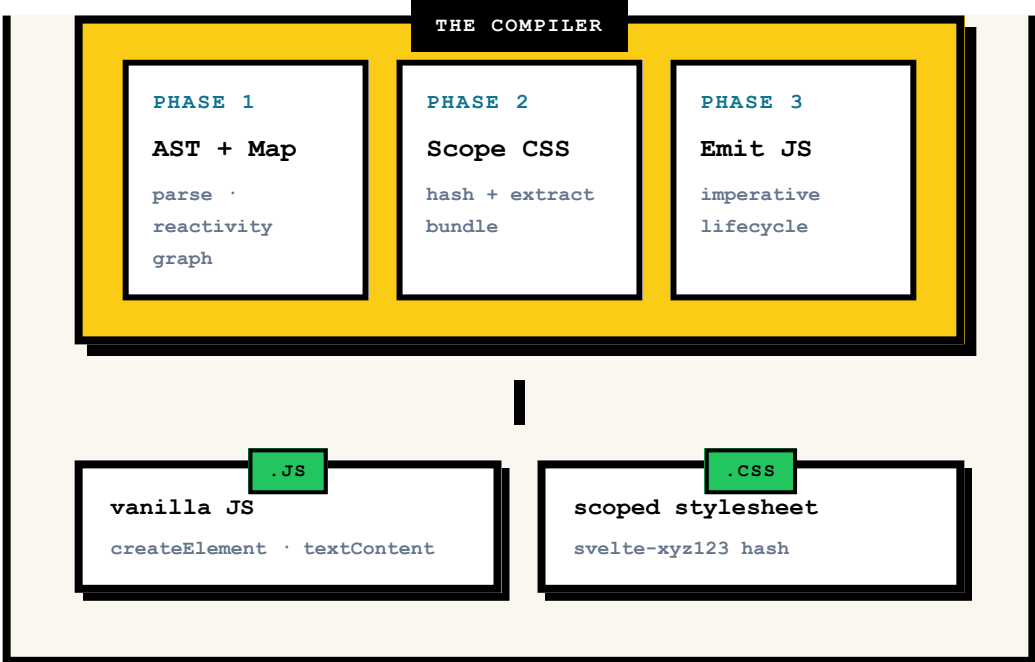
INSIDE THE BUILD STEP



Counter.svelte

source component





The Performance Benefits of a Compiler

Q03

SVELTE 5 PERFORMANCE BUNDLE SIZE DEPENDENCY GRAPH

INTERVIEW QUESTION

Question. What are the primary performance benefits of shifting work from runtime to compile time?

ANSWER.

1. Minimal Bundle Size (The Network Benefit)

- **No Runtime Library:** because there is no heavy runtime library or Virtual DOM engine to ship, you only send the specific vanilla JavaScript your application needs.
- **Faster Metrics:** this results in drastically smaller bundles, meaning faster downloads and quicker "Time to Interactive" metrics for users on slow networks.

2. Surgical Updates (The CPU Benefit)

- **Dependency Graph:** Svelte calculates a Dependency Graph (or Reactivity Graph) at compile time using the Abstract Syntax Tree (AST).
- **Pre-calculated Mind:** the framework already knows exactly which DOM nodes depend on which pieces of state before the app even loads in the browser.

No Virtual DOM Diffing

- **Eliminating Work:** when data changes, Svelte doesn't need to

generate a Virtual DOM or perform expensive tree-diffing algorithms.

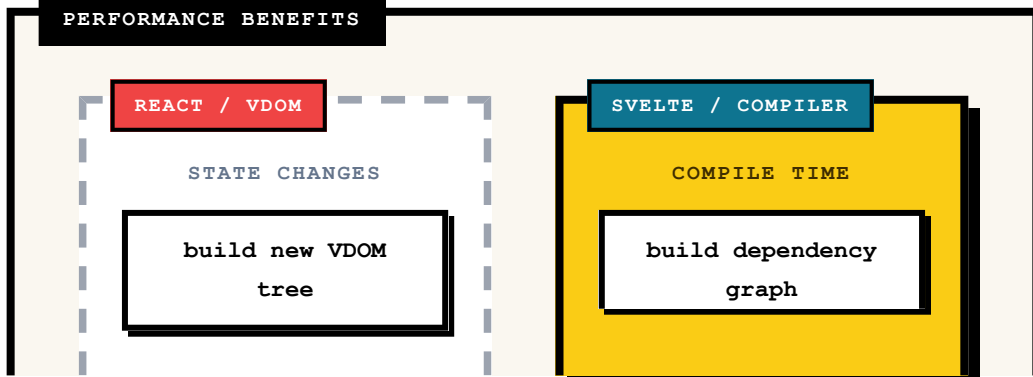
- **Direct Execution:** it simply follows the pre-calculated Dependency Graph and updates the exact target DOM node instantly.

Bundle_Comparison.js

```
function react_style_update(oldTree, newTree) {  
  // ✗ a Virtual DOM must diff trees on every change, wasting  
  CPU  
  return reconcile(oldTree, newTree);  
}  
  
function svelte_style_update(new_count) {  
  // ✓ the pre-calculated graph runs the update INSTANTLY  
  text_node.data = new_count;  
}
```

❗ **Summary.** Shifting work to compile time provides two massive performance benefits: smaller bundle sizes (because you don't ship a framework engine over the network) and highly efficient CPU usage (because surgical updates replace expensive Virtual DOM diffing at runtime).

PERFORMANCE BENEFITS





Syncing Variables with Props

Q49

SVELTE 5 REACTIVITY PROPS RUNES

INTERVIEW QUESTION

Question. How do you initialize a reactive local variable that dynamically syncs with an incoming prop using Runes?

ANSWER.

Why `$state` fails here

- **One-time read:** assigning a prop directly to a `**$state**` rune (like `let count = $state(initialCount)`) only captures the initial value. The `$state` rune initializes once during component creation and creates an independent copy.
- **Severed connection:** if the parent component updates the prop later, the child's `$state` variable **will not update**, leaving the UI out of sync with the parent's data.

The correct approach — `$derived`

- **Continuous sync:** to keep a local variable synced with a prop, declare it with the `**$derived**` rune.
- **Automatic recalculation:** anything read synchronously inside the `$derived` expression is tracked as a dependency. When the prop changes, the `$derived` variable is marked as **dirty** (stale) and automatically recalculates, keeping the child component perfectly in sync with the parent.

Optimistic overrides

- **Temporary assignment:** as of Svelte 5.25, `$derived` variables are no

longer strictly read-only. You can reassign them directly to create **optimistic UI** — updating the screen instantly before the server confirms the action.

- **Washed away by truth:** when the server responds and the parent eventually passes down a new prop, the `$derived` expression recalculates. The temporary override is safely replaced by the new **source of truth**.

Post.svelte

```
<script>
  let { post, like } = $props();

  // 1. Syncs automatically with post.likes whenever it
  changes
  let likes = $derived(post.likes);

  async function handleLike() {
    // 2. Optimistic override: instantly update the UI locally
    likes += 1;

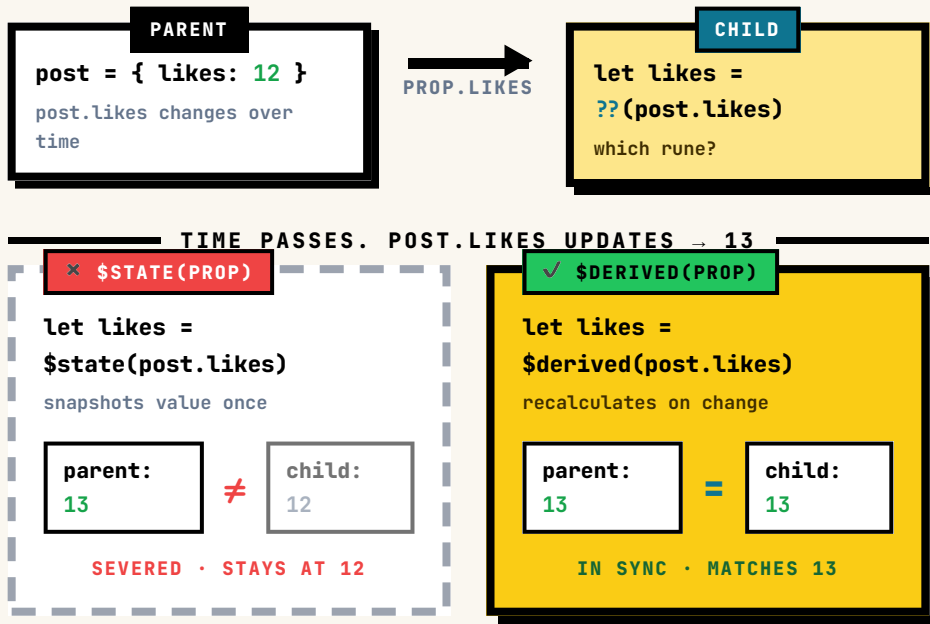
    try {
      await like(); // Tell the server
    } catch {
      likes -= 1; // Rollback if the network request fails
    }
  }
</script>

<button onclick={handleLike}>
  ❤️ {likes}
</button>
```

i Summary. Use `$derived(prop)` rather than

`$state(prop)` to create a reactive local variable that stays synced with an incoming prop. The `$state` rune only reads the prop once during initialization, whereas `$derived` continuously recalculates. You can temporarily reassign the `$derived` variable for optimistic UI updates before the new prop arrives.

TWO WAYS TO HANDLE A PROP



5.25+ BONUS · OPTIMISTIC OVERRIDE

